Guia Avançado: Variantes de Listas, Pilhas e Filas

Especificações Técnicas, Estruturas Duplamente Encadeadas, Circulares e Filas de Prioridade em Java

1. LISTA DUPLAMENTE ENCADEADA (DOUBLY LINKED LIST)

Diferencial: Cada nó possui dois ponteiros (`anterior` e `proximo`). Permite navegação bidirecional (para frente e para trás) e remoção O(1) quando o nó já é conhecido.

```
public class ListaDuplamenteEncadeada<T> {
    private static class No<T> {
        T dado;
        No<T> anterior;
        No<T> proximo;
        No(T dado) { this.dado = dado; }
    }

    private No<T> cabeca;
    private No<T> cauda;

    public void inserirInicio(T dado) {
        No<T> novo = new No<>(dado);
        if (cabeca == null) {
            cabeca = cauda = novo;
        } else {
            novo.proximo = cabeca;
            cabeca.anterior = novo;
            cabeca = novo;
        }
    }

    public void inserirFim(T dado) {
        No<T> novo = new No<>(dado);
        if (cauda == null) {
            cabeca = cauda = novo;
        } else {
            cauda.proximo = novo;
            novo.anterior = cauda;
            cauda = novo;
        }
    }

    public T removerFim() {
        if (cauda == null) throw new IllegalStateException("Lista vazia");
        T item = cauda.dado;
        if (cabeca == cauda) {
            cabeca = cauda = null;
        } else {
            cauda = cauda.anterior;
            cauda.proximo = null;
        }
        return item;
    }
}
```

2. LISTA CIRCULAR SIMPLES (CIRCULAR LINKED LIST)

Diferencial: O ponteiro `proximo` do último nó aponta de volta para o primeiro nó (`cabeca`). Não existem ponteiros `null` nas extremidades.

```
public class ListaCircularSimples<T> {
    private static class No<T> {
        T dado;
        No<T> proximo;
        No(T dado) { this.dado = dado; }
    }

    private No<T> cauda = null; // Apontar para a cauda facilita inserção no início e fim em O(1)

    public void inserir(T dado) {
        No<T> novo = new No<>(dado);
        if (cauda == null) {
            cauda = novo;
            cauda.proximo = cauda;
        } else {
            novo.proximo = cauda.proximo;
            cauda.proximo = novo;
            cauda = novo;
        }
    }

    public void imprimir() {
        if (cauda == null) return;
        No<T> atual = cauda.proximo;
        do {
            System.out.println(atual.dado);
            atual = atual.proximo;
        } while (atual != cauda.proximo);
    }
}
```

3. DEQUE - FILA DUPLAMENTE TERMINADA (DOUBLE-ENDED QUEUE)

Conceito: Permite inserção e remoção em ambas as extremidades (Início e Fim) com complexidade **O(1)**. Funciona simultaneamente como Pilha e Fila.

```
public class DequeEncadeado<T> {
    private static class No<T> {
        T dado;
        No<T> ant, prox;
        No(T dado) { this.dado = dado; }
    }
    private No<T> inicio, fim;

    public void addFirst(T dado) {
        No<T> novo = new No<>(dado);
        if (inicio == null) inicio = fim = novo;
        else { novo.prox = inicio; inicio.ant = novo; inicio = novo; }
    }

    public void addLast(T dado) {
        No<T> novo = new No<>(dado);
        if (fim == null) inicio = fim = novo;
        else { novo.ant = fim; fim.prox = novo; fim = novo; }
    }

    public T removeFirst() {
        if (inicio == null) throw new IllegalStateException("Deque vazio");
        T item = inicio.dado;
        inicio = inicio.prox;
        if (inicio == null) fim = null;
        else inicio.ant = null;
        return item;
    }

    public T removeLast() {
        if (fim == null) throw new IllegalStateException("Deque vazio");
        T item = fim.dado;
        fim = fim.ant;
        if (fim == null) inicio = null;
        else fim.prox = null;
        return item;
    }
}
```

4. FILA CIRCULAR ESTÁTICA SOBRE ARRAY (CIRCULAR ARRAY QUEUE)

Otimiza o reuso de espaço em arrays fixos reaproveitando índices desocupados por dequeue utilizando operador módulo %.

```
public class FilaCircularArray {
    private int[] dados;
    private int inicio = 0, fim = 0, tamanho = 0, capacidade;

    public FilaCircularArray(int capacidade) {
        this.capacidade = capacidade;
        this.dados = new int[capacidade];
    }

    public boolean enqueue(int item) {
        if (tamanho == capacidade) return false; // Fila cheia
        dados[fim] = item;
        fim = (fim + 1) % capacidade; // Avanço circular
        tamanho++;
        return true;
    }

    public int dequeue() {
        if (tamanho == 0) throw new IllegalStateException("Fila vazia");
        int item = dados[inicio];
        inicio = (inicio + 1) % capacidade; // Avanço circular
        tamanho--;
        return item;
    }
}
```

5. FILA DE PRIORIDADE (PRIORITY QUEUE / MIN-HEAP)

Os elementos são removidos na ordem de prioridade (menor/maior valor) em vez de ordem de chegada. Implementada sobre um Binary Heap em Array.

```java
public class FilaPrioridadeMinHeap {
    private int[] heap;
    private int tamanho = 0;

    public FilaPrioridadeMinHeap(int cap) { heap = new int[cap]; }

    public void inserir(int val) {
        if (tamanho == heap.length) throw new IllegalStateException("Cheio");
        heap[tamanho] = val;
        subir(tamanho++);
    }

    public int extrairMin() {
        if (tamanho == 0) throw new IllegalStateException("Vazio");
        int min = heap[0];
        heap[0] = heap[--tamanho];
        descer(0);
        return min;
    }

    private void subir(int i) {
        while (i > 0 && heap[i] < heap[(i - 1) / 2]) {
            int p = (i - 1) / 2;
            int t = heap[i]; heap[i] = heap[p]; heap[p] = t;
            i = p;
        }
    }

    private void descer(int i) {
        while (2 * i + 1 < tamanho) {
            int j = 2 * i + 1;
            if (j + 1 < tamanho && heap[j + 1] < heap[j]) j++;
            if (heap[i] <= heap[j]) break;
            int t = heap[i]; heap[i] = heap[j]; heap[j] = t;
            i = j;
        }
    }
}
```

MATRIZ COMPARATIVA DE ESTRUTURAS AVANÇADAS

Estrutura	Inserção	Remoção	Busca	Vantagem Principal
Lista Duplamente Encadeada	O(1) extremidades	O(1) extremidades	O(N)	Navegação bidirecional e remoção de nós conhecidos em O(1).
Lista Circular	O(1) inicio/fim	O(1) inicio	O(N)	Estrutura contínua perfeita para algoritmos de escalonamento (Round-Robin).
Deque (Double-Ended Queue)	O(1) em ambas pontas	O(1) em ambas pontas	O(N)	Flexibilidade total (atua como Pilha e Fila simultaneamente).
Fila Circular (Array)	O(1)	O(1)	O(N)	Alta eficiência de memória sem alocação dinâmica de ponteiros.
Fila de Prioridade (Heap)	O(log N)	O(log N) extração min/máx	O(N)	Acesso instantâneo ao elemento de maior/menor prioridade.